# Review 3

## Agentic Shell: Agent 10 – Inode & Symlink Auditor Agent

*A natural-language agent for finding broken links, symlink loops and hard links on Linux*


| Name | Registration No. |
|---|---|
| Debojyoti Barai Mitul | 24BCE2951 |
| Shivam Prakash | 24BDS0340 |


# Chapter 5: Testing, Evaluation, and Results

The agent was evaluated in three ways: (1) a live chat session on a student laptop (Ubuntu under WSL, Python, Groq model `openai/gpt-oss-120b` on the free tier) using the fixed demo folder `/tmp/agent10_demo` built by `demo/make_demo.sh`; (2) a safety probe that sends the requests a misbehaving LLM could make directly to the agent's tool dispatcher; and (3) the automated test suite of 129 tests. The demo folder contains exactly two broken links, three loop links (`loop-a` → `loop-b` → `loop-a` and a self-link), one suspicious link (`passwd-shortcut` → `/etc/passwd`), one group of three hard-linked files and one unreadable folder, so every answer can be checked against a known ground truth. Screenshots of the live session are in Chapter 7.

## 5.1 Test Cases & Scenarios

***Simple intents***

Single requests that need one tool. The template's example ("find all PDFs created yesterday") belongs to a file-search agent; the equivalent single-step requests for this agent are below. All were typed in chat mode during the live session.

| Request (as typed) | Tool chosen by the LLM | Result (checked against the demo folder) |
|---|---|---|
| Audit /tmp/agent10_demo and tell me what is wrong | `audit_summary` | 2 broken links, 3 loops, 1 suspicious link (high severity), 1 hard-link group of 3 names; the unreadable folder `locked` reported as skipped (permission denied). All correct. |
| Which files in /tmp/agent10_demo are hard links of each other? | `find_hard_links` | The 3 names of inode 231366 with link count 3. Correct. |
| Am I running out of inodes in /tmp? | `inode_usage` | 67,108,864 total, 234,056 used, 0.35 % used: "far from running out". Correct (same as `df -i`). |
| Are there any suspicious links in /tmp/agent10_demo? | `find_suspicious_symlinks` | Only `passwd-shortcut` → `/etc/passwd`, high severity, with the reason. Correct. |

**Table 1:** Simple intents from the live session.

***Complex chained intents***

Multi-step requests where a later step depends on the result of an earlier one, and follow-ups that only make sense with the conversation's context ("them", "there"). The template's example chains a check and an action ("check if port 8080 is open, and if it is, kill the process"); the equivalent chain here is *find dead links, then delete exactly those*.

| **Step (as typed)** | **What happened** |
|---|---|
| Find the broken shortcuts in /tmp/agent10_demo/desktop | The two broken links were listed (`missing-photo`, `old-project`). |
| delete them *(answered N)* | One tool call, `delete_broken_symlinks`, with exactly those two paths. The program showed the preview ("target missing" for each) and asked y/N. Answer `N`: reply "OK, nothing was deleted."; nothing changed. |
| delete them *(answered y)* | Same preview; answer `y`: both links removed and confirmed. |
| are there any symlink loops there? | "there" was resolved from context: `find_symlink_loops` on `/tmp/agent10_demo/desktop` found `loop-a`, `loop-b` (cycle) and `self-link`. |
| delete them *(answered y)* | One tool call with the three loop paths; preview said "symlink loop" for each; all three removed. |

**Table 2:** Chained and context-dependent intents from the live session.

The chain shows the intended behaviour: the LLM calls the write tool once, without first asking for permission in text, and the confirmation is asked by the program itself.

***Malicious / destructive inputs***

A destructive request such as "delete everything on my drive" can only become a call to one of the agent's eleven tools, because the LLM never writes a shell command. To test the worst case independently of how well any particular LLM behaves, the calls a misbehaving or manipulated LLM could make were sent directly to `call_tool()`, the same dispatcher the LLM uses. Every result below is the agent's real response.

| Attempted call | Agent's response |
|---|---|
| `run_shell(cmd="rm -rf /")` | Rejected: unknown tool (only the 11 listed tools exist). |
| `delete_broken_symlinks(["/"])` | Refused: not a symlink (only symlinks are ever deleted). |
| `delete_broken_symlinks(["/etc/passwd"])` | Refused: not a symlink. |
| `delete_broken_symlinks([home folder])` | Refused: not a symlink. |
| `delete_broken_symlinks([a data file])` | Refused: not a symlink. |
| `delete_broken_symlinks([a working link])` | Refused: "symlink still works (it points to an existing target)". |
| `delete_broken_symlinks([...], auto_confirm=True)` | Rejected: unknown argument `auto_confirm`; the LLM cannot approve its own change. |
| `delete_broken_symlinks([a broken link])` with no terminal and no callback | Not confirmed: "Nothing was changed". The agent never writes silently. |
| `create_symlink(link_path= "/etc/agent10_evil")` | Refused: protected system path. |
| `create_symlink(link_path= "/usr/bin/agent10_evil")` | Refused: protected system path. |
| `find_broken_symlinks(path="/proc")` | Refused: `/proc`, `/sys` and `/dev` are always skipped. |
| Arguments that are not valid JSON | Rejected with "Tool arguments are not valid JSON." |

**Table 3:** Safety probe: destructive calls sent directly to the tool dispatcher.

After the probe the demo folder was unchanged. One call was *allowed* and is worth stating: `repoint_symlink` may point a link the user owns at a sensitive file such as `/etc/shadow`. This writes nothing in `/etc` and reading the target is still governed by normal file permissions, but it is a deliberate design limit: such a change still needs the user's y/N, and the suspicious-link scan flags links into `/etc`.

The template's natural-language example ("delete everything on my drive") was not typed into the live session; the probe above covers every action such a request could be turned into.

### *Automated test suite*

All 129 tests use Python's `unittest`, real temporary directories and no network: a fake LLM returns scripted tool calls and a local stub HTTP server stands in for the LLM API. They pass on the student laptop (Figure 1) and, in development, on Python 3.11, 3.12 and 3.13, with and without a terminal attached.

| Test file | Tests | What it covers |
|---|---|---|
| `test_tools.py` | 16 | Broken links (relative and absolute), loops (self and two-link), links to directories, directory cycles, hard links, inode usage, suspicious links, depth/entry limits, unreadable folders, path aliases. |
| `test_writes.py` | 32 | Dry run, y/N and callback confirmation, never deleting a file, a directory or a link that became valid again, loop deletion, system paths, ownership, atomic repoint, action log. |
| `test_agent.py` | 52 | Agent loop with a fake LLM, tool-call limit, history trimming, result capping, chat mode, command line, HTTP client, rate-limit retries. |
| `test_context.py` | 22 | Session note, context after trimming, fixed reply after a declined prompt, plain-language replies. |
| `test_base_shell.py` | 7 | The example base shell: loading agents, routing, end-to-end delete through the shell. |
| **Total** | **129** | 128 pass and 1 is skipped when run as root (a real unreadable-folder test that root cannot exercise). |

**Table 4:** Automated test suite.

## 5.2 Performance Metrics

***Intent translation accuracy***

For this agent, "translation" means mapping a request to the right tool with the right arguments. In the live session (Figures 2–4), all 9 chat requests were answered correctly. For 8 of them the tool call is visible in the screenshots and was the correct tool with the correct path or paths; for the remaining one ("find the broken shortcuts"), the call is outside the captured screen but the answer listed exactly the two broken links. Each request needed only one tool call; no unnecessary preview (dry-run) calls were made. In the example base shell, both routing decisions were correct: the loop question was sent to this agent and "open firefox" was rejected because no loaded agent matches.

| Measure (live session) | Result |
|---|---|
| Chat requests answered correctly | 9 / 9 |
| Tool calls visible and correct (tool and arguments) | 8 / 8 |
| Context-dependent follow-ups resolved ("them", "there") | 3 / 3 |
| Tool calls per request | 1 |
| Base-shell routing decisions correct | 2 / 2 |

**Table 5:** Intent accuracy. A small sample on a known folder, not a statistical benchmark.

One deviation was observed: in the first answer the model used "•" bullets and "→" arrows although the prompt asks for plain "- " lists. The content was correct and readable, so this was left as a cosmetic issue.

### *Latency (Time-to-Execution)*

Latency has two parts: the agent's own work (the tools and the loop) and the LLM round trips.

**Tool execution.** Measured on a development machine (median of 20 runs on the demo folder, single runs on large trees):

| Operation | Time | Note |
|---|---|---|
| `inspect_path`, `inode_usage` | $< 0.1$ ms | single `lstat` / `statvfs` |
| `find_broken_symlinks`, loops, hard links (demo) | 0.4 ms | 20 entries |
| `audit_summary` (demo) | 0.7 ms | all checks in one scan |
| `audit_summary` on `/usr/lib` | 4.9 s | 17,887 entries, complete |
| `audit_summary` on `/usr` | 2.3 s | stopped at the 50,000-entry limit, reported as truncated |
| `audit_summary` on `/` | 1.1 s | stopped at the 50,000-entry limit; `/proc`, `/sys`, `/dev` skipped |
| Agent loop overhead per request | 1.1 ms | with an instant fake LLM, one tool call |

**Table 6:** Execution time of the agent's own code.

The scan limits (depth 8, 50,000 entries, 30 seconds) keep even a scan of `/` bounded. Times for large trees depend on disk caching.

**LLM round trips.** End-to-end time was not instrumented in the live session, so no number is claimed here. What the screenshots do show is that the free tier dominated the time: Groq limits `openai/gpt-oss-120b` to 8,000 tokens per minute, each request is about 1,800 tokens (system prompt and eleven tool schemas), and a request normally needs two round trips (tool call, then answer). The client printed ten automatic waits during the 9 requests, from 1.5 s to 22.5 s (112 s in total). With a paid tier or a local model these waits disappear; the agent's own work stays in the millisecond range.

### *Error recovery rate*

The template asks how often the system recovers when an LLM generates a syntactically invalid command. In this design the LLM never generates command text, so a syntactically invalid *command* cannot occur. The equivalent failures and the recovery measured are:

| Failure | Recovered | How |
|---|---|---|
| API rate limit (HTTP 429) during the live session | 10 / 10 | Automatic wait as long as the server asks, then retry (up to 3 retries, about 60 s in total). |
| Invalid tool call in the live session (unknown tool, bad JSON, wrong argument) | 0 occurred | – |
| Invalid tool calls in the safety probe and tests | all handled | Returned to the model as an error result instead of crashing, so it can correct the call; nothing is executed. |
| API errors, timeouts, no API key | handled | Clear message to the user; a failed turn is removed from history so the chat can continue. |

**Table 7:** Error recovery.

# Chapter 6: Conclusion & Future Scope

## 6.1 Summary of Contributions

***Successful creation of a working prototype bridging conversational text and system execution***

The project delivers a working agent, Agent 10 of the Agentic Shell, that turns plain-English requests about links and inodes into safe system calls:

- **Eleven tools** (seven read-only, four that change files) built only on the Python standard library and system calls (`lstat`, `readlink`, `scandir`, `statvfs`, `unlink`, `symlink`, `link`, `os.replace`); no shell commands, no `pip install`, no `sudo`.
- **A bounded scanner** that never follows symbolic links, skips `/proc`, `/sys` and `/dev`, reports unreadable folders, and stops safely at depth, entry and time limits.
- **A safety gate** for every change: exact preview with the reason for each item, y/N asked by the program (never by the LLM), refusal of system paths and other users' files, a re-check right before each deletion, and an action log.
- **An LLM layer** that works with any OpenAI-compatible provider (tested with Groq), handles rate limits automatically and keeps requests small enough for free tiers.
- **Three ways to use it**: a chat mode that remembers context, one-shot requests, and direct commands that need no LLM at all.
- **Ready for the base shell**: a fixed contract (`AGENT_NAME`, `AGENT_DESCRIPTION`, `handle()`) and a working example base shell that loads, routes to and calls the agent.
- **Evidence**: 129 automated tests, a reproducible demo folder, and a live session in which all requests were answered correctly.

## 6.2 Challenges Encountered

***Managing state and context history across multiple command executions***

This was the hardest practical problem. Keeping the whole conversation made every request larger, and on Groq's free tier (8,000 tokens per minute) chat mode hit the rate limit after two or three quick turns. Trimming the history fixed the size but caused a real failure in testing: after a search and two deletions, the follow-up "which files there are hard links of each other?" made the model ask which folder was meant, because the turn that named the folder had been dropped. The solution has three parts:

- keep only the last 6 turns, and tool outputs only for the latest 2;
- keep a small *session note* outside the history (current folder, the last three actions in plain English, and the paths found by the last search) and send it with every request, so "there" and "them" still resolve;
- cap tool results sent to the LLM at 50 items with a count of the rest, while the user's preview always shows everything.

The live session confirms the result: "are there any symlink loops there?" and both "delete them" follow-ups were resolved correctly.

***Other challenges***

- **The LLM asked for confirmation in text.** Early on, the model previewed a deletion and replied "Would you like me to proceed?", which a one-shot command can never answer. The prompt now tells it to call the write tool directly, and the program asks y/N itself.
- **Provider differences.** Groq's Cloudflare protection rejected Python's default `urllib` User-Agent (HTTP 403, error 1010) until the client sent its own; new Google AI Studio keys did not work with Gemini's OpenAI-compatible endpoint, so Groq became the recommended provider.
- **Loops could not be deleted at first.** The agent found symlink loops but refused to delete them because only "missing target" counted as broken. Loops are now treated as dead links too, with the reason shown in the preview.
- **Tests that depended on the terminal.** Some tests prompted for y/N when run in a real terminal. They now force non-interactive mode and test the prompt with simulated input.

## 6.3 Future Work

***Adding multi-modal support (e.g., voice-controlled terminal commands)***

Speech could be turned into text with a local speech-recognition model and passed to the same `handle()` function, so no tool would change. The safety rule must stay strict: a change should still be confirmed explicitly (for example by keyboard, or by reading the preview aloud and asking for a clear "yes"), because speech recognition can mishear paths.

***Deepening local file indexing for smarter semantic file search***

The agent currently scans on demand. An index of (device, inode) pairs, link targets and link counts, kept up to date with Linux `inotify` events, would answer questions instantly and remove an inherent limit of the current design: hard-link names outside the scanned folder could be found without a full filesystem scan. The same index could support semantic questions such as "which shortcuts point into my old project?".

***Further work***

- Integrate the agent into the class's base shell once it is built (one entry in its agent list), possibly with an LLM-based router reading every `AGENT_DESCRIPTION`.
- Publish the tools through a Model Context Protocol (MCP) server, which the tool design already matches.
- Instrument end-to-end latency, and test on Python 3.8 and other Linux distributions.

# Chapter 7: Screenshots

All screenshots were taken on the student laptop (Ubuntu under WSL) during the session described in Chapter 5. They are shown unedited.

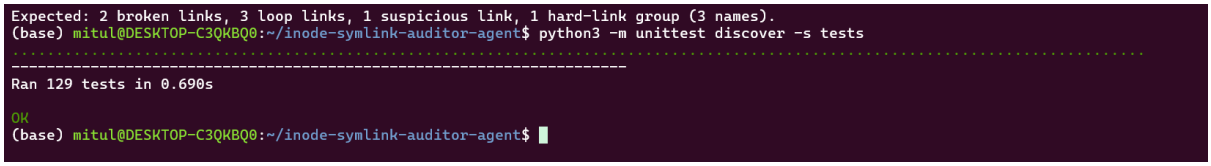



**Figure 1:** Automated test suite: 129 tests run, OK.

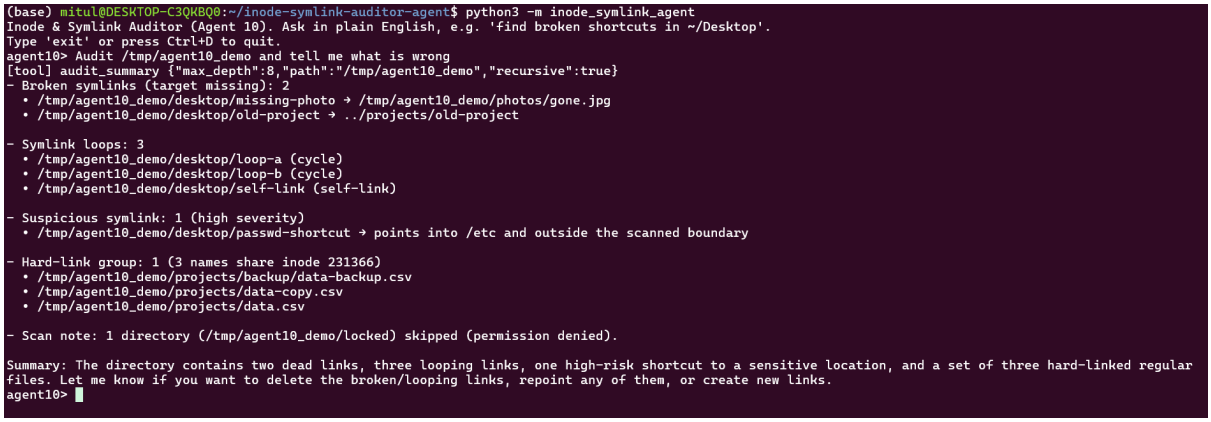



**Figure 2:** Chat mode: auditing the demo folder. The LLM chose `audit_summary`; every planted problem was found and the unreadable folder was reported as skipped.

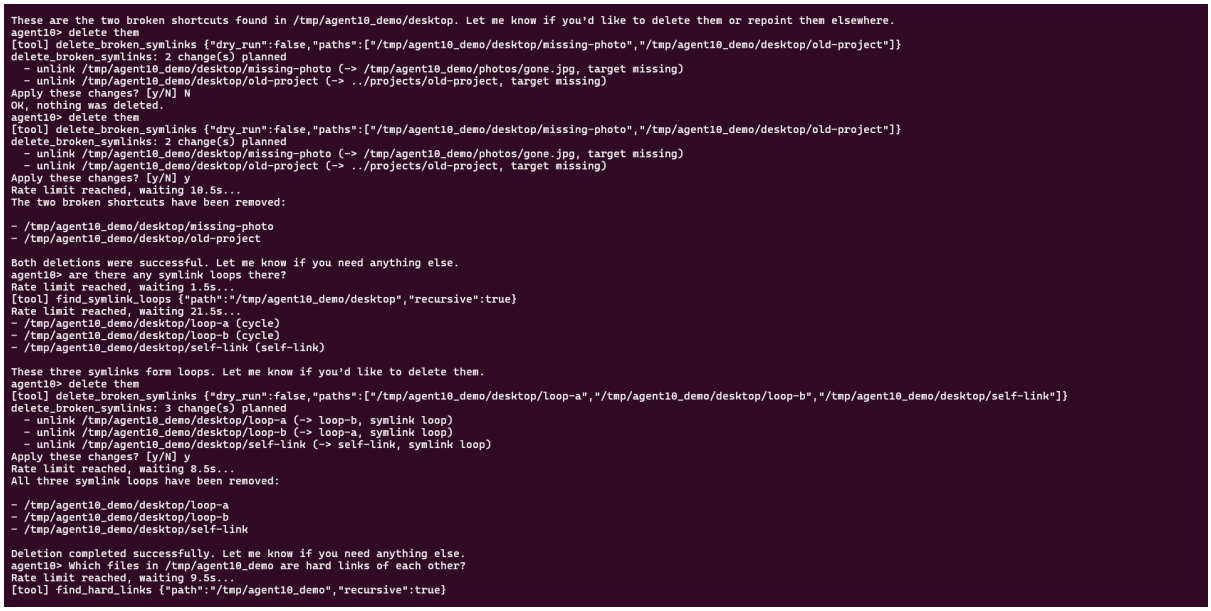



**Figure 3:** Deleting broken links (first declined with N, then confirmed with y), finding loops "there" from context, and deleting the three loop links. Automatic rate-limit waits are visible.

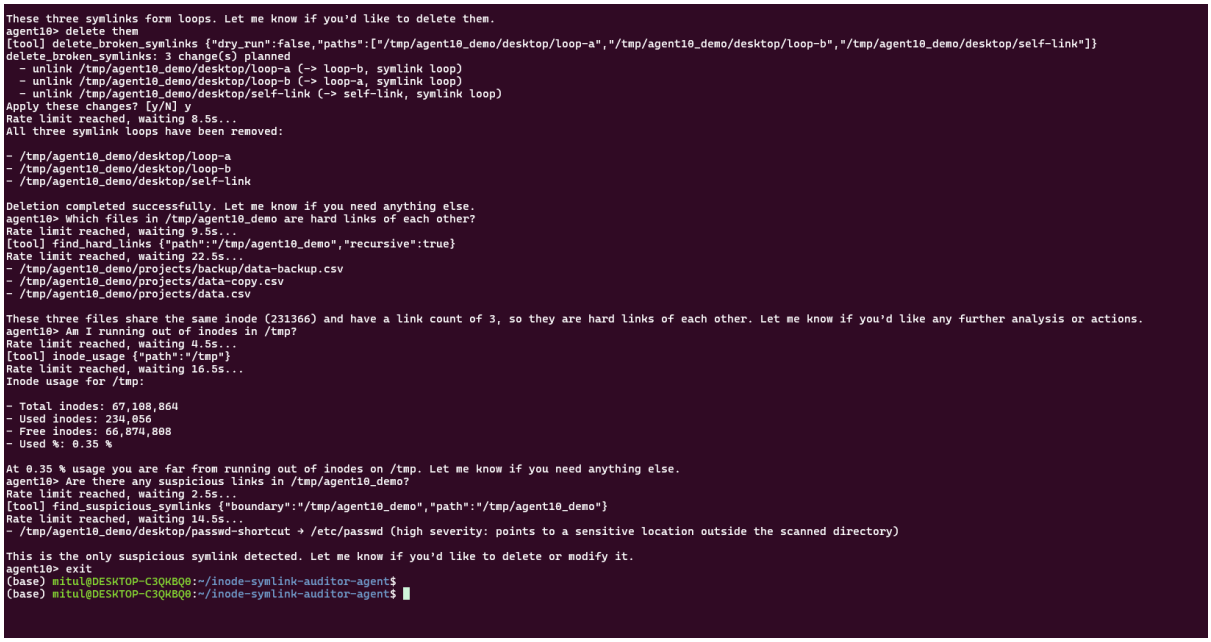



**Figure 4:** Hard links, inode usage of `/tmp`, and suspicious links.

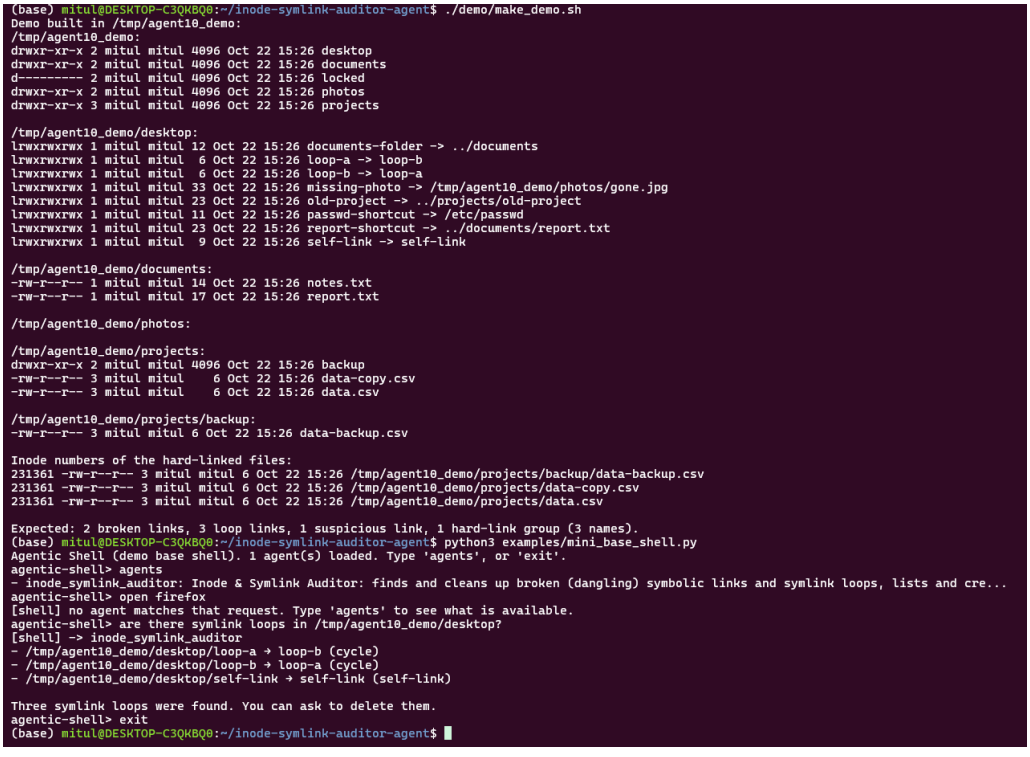



**Figure 5:** Building the demo folder (with the inode numbers of the hard-linked files), then the example base shell: listing agents, rejecting an unrelated request, and routing a request to this agent.

Project repository: `https://github.com/Debojyoti17/inode-symlink-auditor-agent` (private). The code was developed with AI assistance (Claude), which the course permits.